

Tools and Function Calling

Week 05

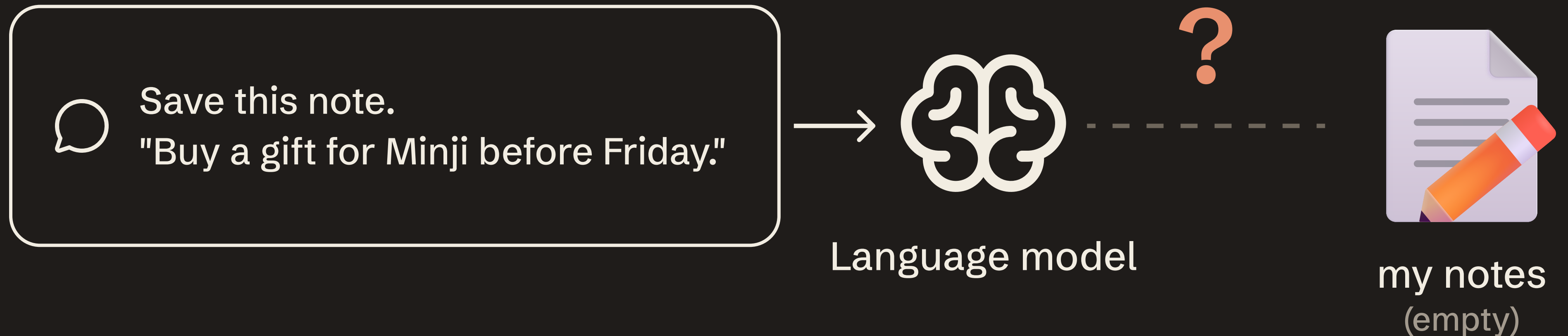
Business Agent Design

Hyungu Kahng

Department of Digital Management, Korea University

hgkahng@korea.ac.kr

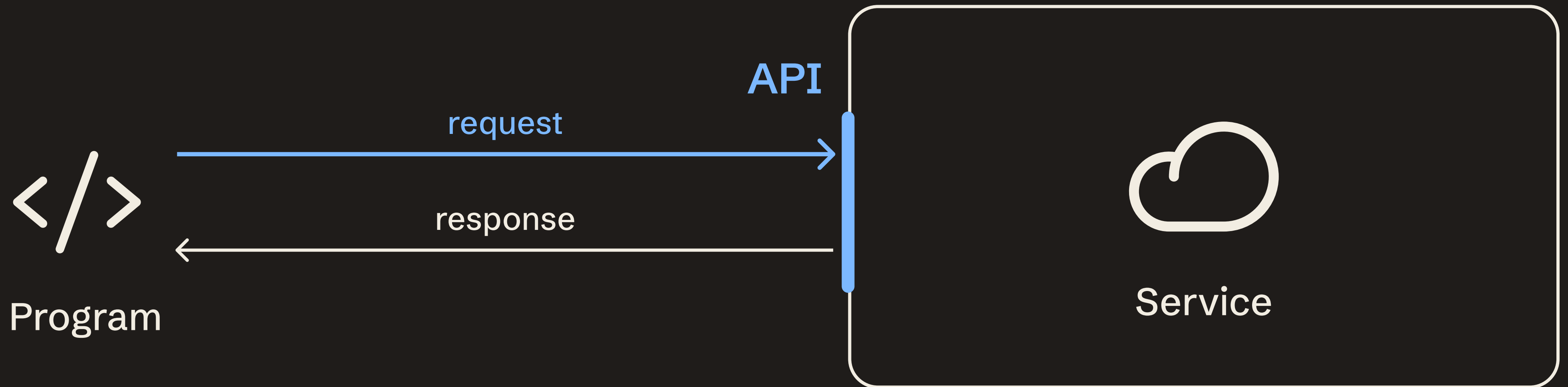
A request without an available action



The model understands the request, but has no way to write the note.

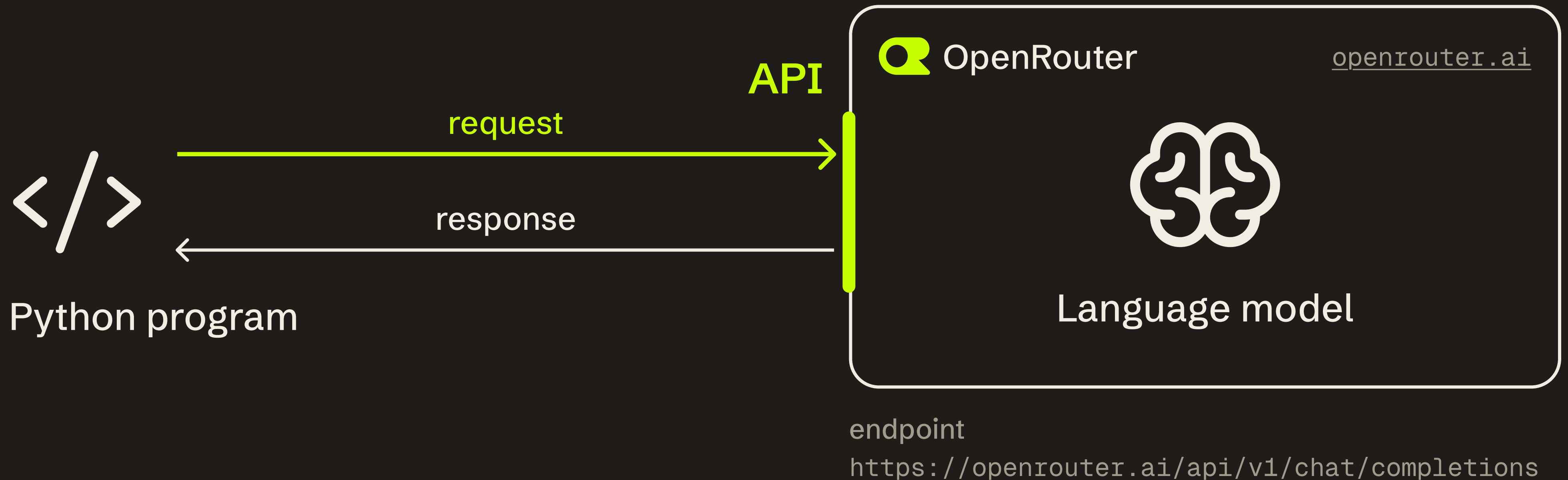
What is an API?

Application Programming Interface



An API defines which requests a service accepts and what it returns.

The OpenRouter API



The Python program uses a language model through the OpenRouter API.

The format of the request and response

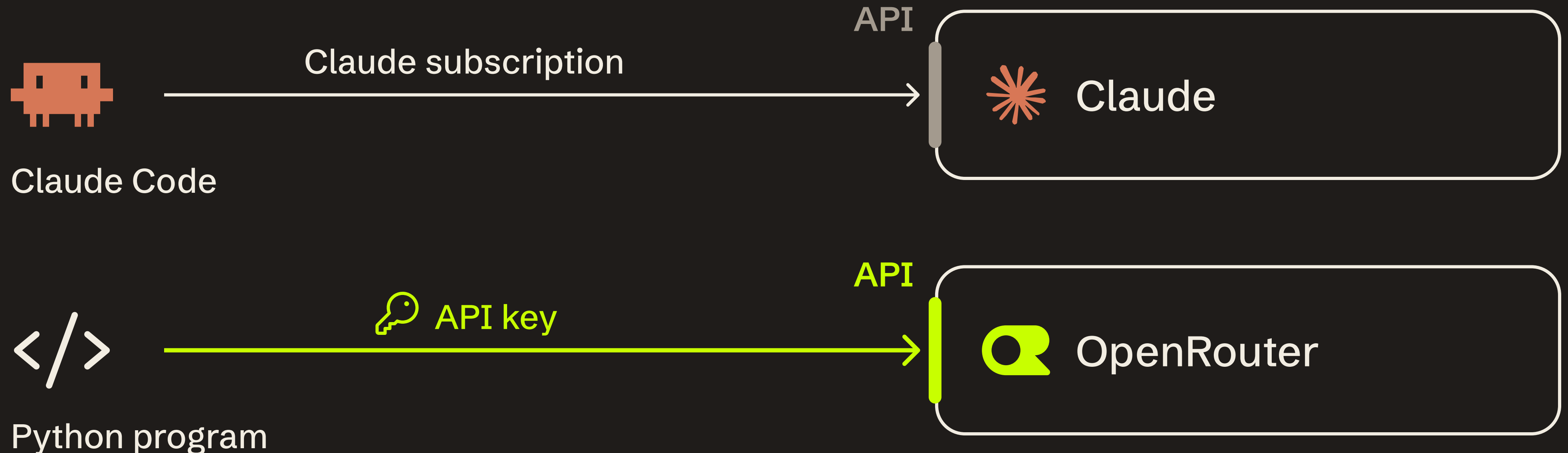
Example

```
request
{
  "model": "<model id>",
  "messages": [{
    "role": "user",
    "content":
      "What is the capital of France?"
  }]
}
```

```
response
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content":
        "The capital of France is Paris."
    }
  }]
}
```

The request sends messages, and the response returns the model's reply.

Subscription and API key



Claude Code uses your Claude subscription.
Your Python program uses an OpenRouter API key.

Keeping the API key secret

📁 business-agent-lab

📄 .gitignore

📄 .env ←

📁 week-05

reads the key

📄 01_model_request.py

📄 02_calculator_call.py

📄 03_assistant.py

📄 tools.py

.env

OPENROUTER_API_KEY=your_key_here

.gitignore

/.env

check from the project root

% git check-ignore .env

.env

The program reads the key from .env, and Git ignores .env.

Run and inspect

1 Run the first script

```
python week-05/01_model_request.py
```

2 Find the endpoint, the messages, and the response

3 Which part of the output comes from the model?

What is a tool?



A tool is a function that the model can ask the program to run.
The tool definition describes the function to the model.

The tool definition

```
tool definition (tools.py, shortened)
"function": {
  "name": "calculator",
  "description":
    "Do one arithmetic operation on two numbers.",
  "parameters": {
    "properties": {
      "operation": {"type": "string",
        "enum": ["add", "subtract",
          "multiply", "divide"]},
      "a": {"type": "number"},
      "b": {"type": "number"}
    },
    ...
  }
}
```

```
function (tools.py, shortened)
def calculator(operation, a, b):
    ...
    if operation == "add":
        result = a + b
    ...
    return result
```

The model reads only the definition. The names in it should match the Python function.

The tool call



user What is 247 times 38?

Captured run

assistant message

```
"tool_calls": [{"id": "call-e7c0364e-...", "type": "function",  
  "function": {"name": "calculator",  
    "arguments": "{\\"operation\\":\\"multiply\\",\\"a\\":247,\\"b\\":38}"}}]
```

A tool call names a function and its arguments. Nothing has run yet.

Execution



The program runs the calculator function. The result comes from Python, not from the model.

Captured run

```
business-agent-lab — zsh

% python week-05/02_calculator_call.py
User: What is 247 times 38?
Assistant message:
...
Requested: calculator(operation="multiply", a=247, b=38)
Press Enter to run the function.
Result: 9386
```

The tool result



messages in the next request

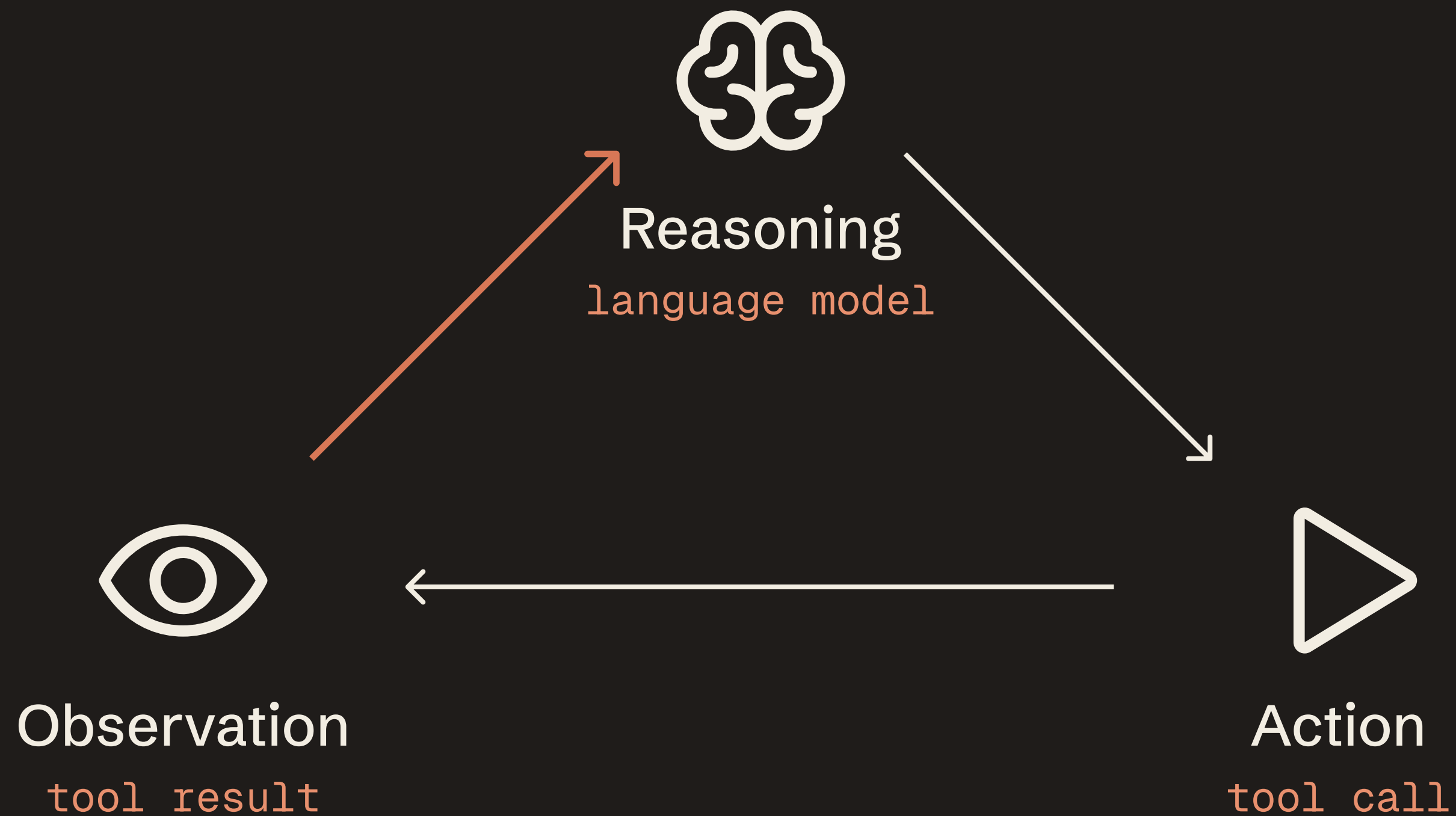
```
{"role": "user", "content": "What is 247 times 38?"}  
{"role": "assistant", "tool_calls": [{"id": "call-e7c0364e- ... ", ... }]}  
{"role": "tool", "tool_call_id": "call-e7c0364e- ... ", "content": "9386"}
```

assistant 247 times 38 equals 9,386.

Captured run

The result goes back with the same id. Then the model writes the answer.

ReAct



An agent uses what it observes after an action to choose the next step.

Demo: The terminal assistant

```
business-agent-lab — zsh

% python week-05/03_assistant.py
Assistant ready. Type exit to quit.
> Hello
assistant    Hello! How can I assist you today?
> _
```

The language model chooses a function, or none. The program runs the chosen function.

tools supplied by the Python program



calculator



read_schedule



search_wikipedia



save_note



convert_currency

exercise

Tools: Schedule reading

Captured run

```
data/schedule.json (fictional, shortened)
[
  ...
  {"date": "2026-10-14", "time": "10:00",
   "title": "Project meeting",
   "place": "Sukwon Hall 204"},
  {"date": "2026-10-14", "time": "18:30",
   "title": "Badminton club",
   "place": "Sejong Campus Gym"},
  ...
]
```

```
business-agent-lab — zsh
> What do I have on October 14?
tool call    read_schedule(date="2026-10-14")
tool result  [{"date": "2026-10-14", "time": "10:00", ...
assistant    On October 14, 2026:
- 10:00: Project meeting in Sukwon Hall 204
- 18:30: Badminton club in Sejong Campus Gym
```

The model does not have your calendar. The tool reads the record.

Tools: Wikipedia search

🔍 Find a Wikipedia page about the Pygmalion effect.

Captured run

```
search_wikipedia result (shortened)
[{"title": "Pygmalion effect",
  "url": "https://en.wikipedia.org/wiki/Pygmalion_effect",
  "excerpt": "The Pygmalion effect is a psychological phenomenon in which high ..."},
... ]
```

The tool returns a source. The assistant interprets it, and you can open it.

Tools: Note saving



```
business-agent-lab — zsh

> Save this note as practice-note: "Buy a gift for Minji before Friday."
tool call    save_note(filename="practice-note", text="Buy a gift for Minji before Friday.")
tool result  saved notes/practice-note.md
assistant    The note has been saved as notes/practice-note.md.

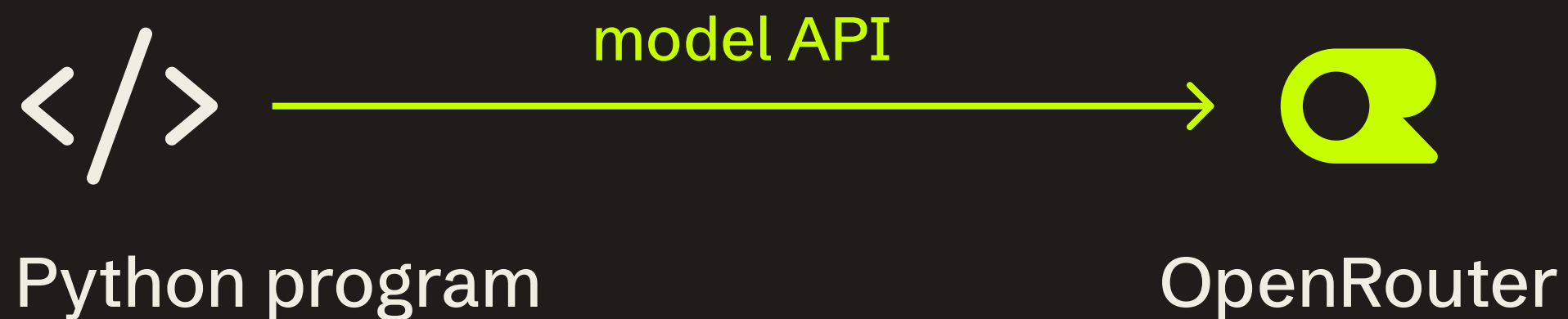
% cat week-05/notes/practice-note.md
Buy a gift for Minji before Friday.
```

The file is the evidence, not a reply that says "saved".

Tools: Currency conversion

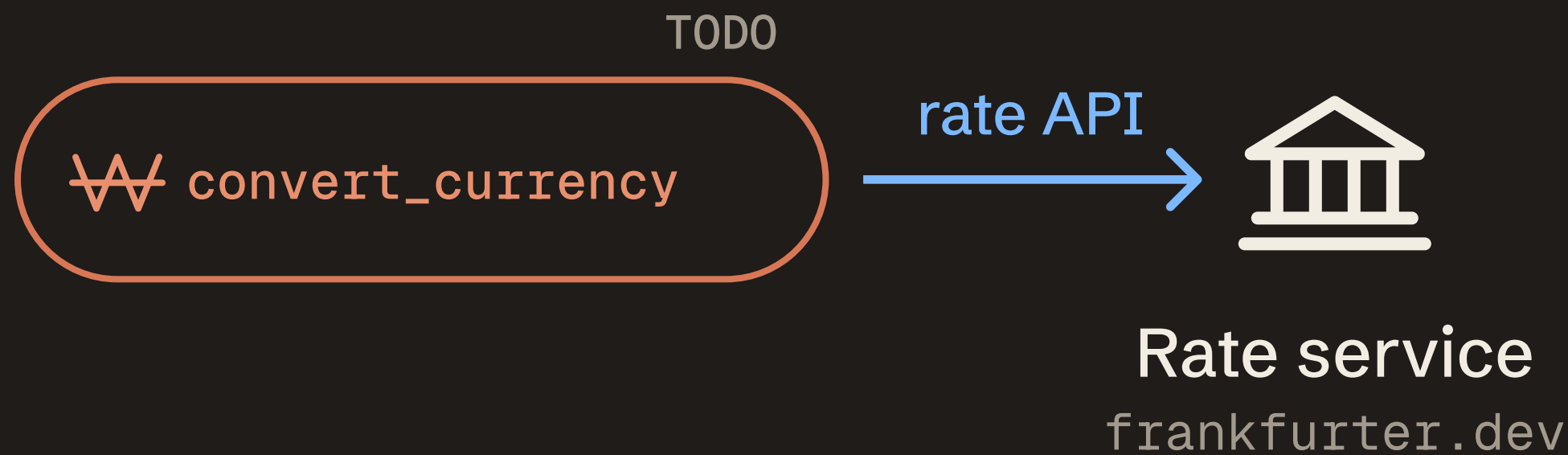
💬 Convert 50 USD to KRW.

Captured run



rate response

```
{"date": "2026-10-05",  
  "base": "USD", "quote": "KRW",  
  "rate": 1347.79}
```



amount × rate

50 × 1347.79 = 67389.5

A dated reference rate, not a bank rate.

The model API and the rate API are two separate connections.

Answer, question, or tool call?

the next step



Hello



Convert 50 dollars



Convert 50 USD to KRW

Answer, question, or tool call?

the next step

Captured run



 a direct answer



 a question
"Convert 50 dollars to which currency?"



 a tool call, `convert_currency`

When a tool fails

```
business-agent-lab — zsh

> What is 10 divided by 0?
tool call      calculator(operation="divide", a=10, b=0)
tool result    error: division by zero
assistant      The calculation failed due to division by zero.
```



division by zero

Captured run

The program catches the error and sends it back as the tool result.

Parts of the assistant

03_assistant.py, shortened

```
messages = [{"role": "system", ...}] # 1
TOOLS = [CALCULATOR, READ_SCHEDULE, ...] # 2

for _ in range(MAX_ROUNDS): # 6
    reply = request_model(messages, TOOLS) # 3
    messages.append({"role": "assistant", ...})
    if not reply.get("tool_calls"):
        trace("assistant", reply.get("content")) # 6
        return
    for call in reply["tool_calls"]:
        result = dispatch(call) # 4
        messages.append(tool_result(call["id"], result)) # 5
```

- 1 messages
- 2 tool definitions
- 3 model request
- 4 dispatch
- 5 tool result
- 6 stopping condition

Find each part in 03_assistant.py, then match it to the trace.

Your extension

1 the function

```
tools.py
def convert_currency(amount, from_currency, to_currency)
```

2 the tool definition

```
tools.py
CONVERT_CURRENCY = {"type": "function",
                    "function": {"name": "convert_currency", ...}}
```

3 the registration

```
03_assistant.py
TOOLS = [ ..., CONVERT_CURRENCY]
FUNCTIONS = { ..., "convert_currency": convert_currency}
```

The new tool needs all three parts before the assistant can use it.

Exercise: currency conversion

- 1** Run the supplied assistant first
`python week-05/03_assistant.py`
- 2** Write the function and its tool definition in `tools.py`
- 3** Add the tool to `TOOLS` and `FUNCTIONS` in `03_assistant.py`
- 4** Convert an amount and read the trace
Finished early? Add a small tool of your own.

Claude Code may help. You explain what the code does.

Peer check

- ☐ Point to the tool arguments in the trace
- ☐ Find the rate, its date, and its source in the tool result
- ☐ Check the answer against $\text{amount} \times \text{rate}$
- ☐ Try a second amount or currency pair
- ☐ Say which API gives the rate, and which gives the reply

Work in groups.

Request, execute, return



The model requests a tool call.

The program runs the function and returns the result.

Appendix: JSON

```
request body (01_model_request.py, shortened)
{
  "model": "nvidia/nemotron-3-super- ... ",
  "messages": [
    {
      "role": "user",
      "content": "What is the capital of France?"
    }
  ]
}
```

```
reading it in Python
body["messages"][0]["content"]
# "What is the capital of France?"
```

JSON in Python

JSON	Python
{ ... }	<code>dict</code>
[...]	<code>list</code>
"text"	<code>str</code>
1347.79	<code>float</code>
true, false	<code>True, False</code>
null	<code>None</code>

JSON is text that holds data.
Python reads it as dicts and lists.

Appendix: Virtual environment

```
business-agent-lab — zsh

% python3 -m venv .venv
% source .venv/bin/activate
(.venv) % pip install -r week-05/requirements.txt
Successfully installed ... python-dotenv-1.2.4 requests-2.34.2
(.venv) % which python
... /business-agent-lab/.venv/bin/python
(.venv) % deactivate
% _
```

```
business-agent-lab/
  .venv/
  week-05/
  .env
  .gitignore
```

.venv holds this project's Python and its packages. Git ignores it.

A virtual environment is a folder with its own Python and packages. Activate it again in each new terminal.

Appendix: Running a Python script

```
business-agent-lab — zsh

% pwd
... /business-agent-lab
% python week-05/01_model_request.py
Endpoint: https://openrouter.ai/api/v1/chat/completions
...
% cd week-05
% python week-05/01_model_request.py
... : can't open file '... /week-05/week-05/01_model_request.py':
[Errno 2] No such file or directory
```

python

runs a Python file

week-05/ ... py

the path, from the current folder

pwd

prints the current folder

Ctrl+C

stops a running program

Run scripts from business-agent-lab/.
The path starts from the current folder.

Appendix: Debugging in VS Code

- 1 Open the business-agent-lab/ folder**
File > Open Folder...
- 2 Choose the .venv interpreter**
Cmd+Shift+P (Windows: Ctrl+Shift+P), then Python: Select Interpreter
- 3 Open week-05/02_calculator_call.py and click left of line 74**
A red dot marks the breakpoint.
- 4 Press F5, then choose Python Debugger and Python File**
Or Run > Start Debugging.
- 5 Press Enter in the terminal**
The program first stops at "Press Enter to run the function."
- 6 Read call in the Variables panel**
The program is paused at line 74. F10 steps, F5 continues, Shift+F5 stops.

Needs the Python extension for VS Code, by Microsoft.

Appendix: Paused at a breakpoint

```
02_calculator_call.py, paused at line 74 (shortened)
72     messages.append({"role": "assistant", ...})
73     for call in reply["tool_calls"]:
74 ●     result = dispatch(call)
75         print("Result:", result)
```

Variables, at line 74

```
call: {'type': 'function', 'id': 'call-e7c0 ... ', ...}
function: {'name': 'calculator', ...}
name: 'calculator'
arguments: '{"operation": "multiply", "a": 247, "b": 38}'
```

debug keys

F5	Start, Continue
F10	Step over
F11	Step into
Shift+F5	Stop

Click left of a line number to set a breakpoint (●). Press F5 to start.

Appendix: if `__name__ == '__main__'`

01_model_request.py, shortened

```
import requests                                # 1 runs first
ENDPOINT = "https://openrouter.ai/ ..."     # 2 runs next
def main(): ...                               # 3 defines main(), does not run it
if __name__ == "__main__":                   # 4 checks the name
    main()                                    # 5 main() runs
```

the value of `__name__` in each file

```
% python week-05/03_assistant.py
```

03_assistant.py	<code>__name__ == "__main__"</code>	<code>main()</code> runs
tools.py (imported)	<code>__name__ == "tools"</code>	
model_api.py (imported)	<code>__name__ == "model_api"</code>	

Python starts at the top. `main()` runs only in the file you run.